Laboratory Exercise 3

Latches, Flip-Flops, and Registers

Name: Monowar Mahmud **Course:** ENCE 4110

1 Objective

The objective of this laboratory was to implement and examine sequential logic circuits using Verilog HDL and Intel Quartus. The implementations included a gated SR latch, a gated D latch, different clock-triggered storage elements, and an eight-bit register interfaced with seven-segment displays on the DE10-Lite FPGA board.

2 Methodology

2.1 Part I: Gated SR Latch

A gated SR latch was implemented in Verilog using continuous assignment statements. The Set (S) and Reset (R) inputs were combined with the clock signal through AND operations to generate the gated signals S_g and R_g. Two cross-coupled NOR gates were then used to establish the feedback required for storing a single bit.

The logic equations were implemented as:

$$R_g = R \cdot Clk, \tag{1}$$
$$S_g = S \cdot Clk, \tag{2}$$
$$Q_a = \overline{R_g + Q_b}, \tag{3}$$
$$Q_b = \overline{S_g + Q_a}. \tag{4}$$

The output Q was assigned from Q_a. The `/* synthesis keep */` directive was applied to the internal feedback signals to preserve them during synthesis.

The implementation was described in `part1.v`. The circuit was designed to retain its previous state when the clock was low and respond to the Set and Reset inputs when the clock was high.

2.2 Part II: Gated D Latch

A gated D latch was implemented using cross-coupled NAND gates. Unlike the SR latch, this circuit used a single data input D, with its complement generating the corresponding Reset input.

The gated signals and feedback equations were defined as follows:

$$R = \overline{D}, \tag{5}$$
$$S_g = \overline{D \cdot Clk}, \tag{6}$$
$$R_g = \overline{R \cdot Clk}, \tag{7}$$

with the feedback equations:

$$Q_a = \overline{S_g Q_b}, \tag{8}$$
$$Q_b = \overline{R_g Q_a}. \tag{9}$$

The design was implemented in `part2.v` using continuous assignments. The synthesis keep directive was included for the internal signals.

The latch was designed to follow the data input while the clock was high and retain the previously stored value when the clock was low. This configuration eliminated the invalid input combination associated with the conventional SR latch.

2.3 Part III: Behavioral Storage Elements

Three storage elements were implemented within the Verilog module `part_iv.v` to examine their different clock-dependent behaviors. All three elements received the same data input D and clock signal.

The first element was a gated D latch, implemented using an `always` block sensitive to changes in D or `Clock`. The output Q_a was updated only when the clock was high, providing level-sensitive operation.

The second element was a positive-edge-triggered D flip-flop. Its output Q_b was updated using an `always @(posedge Clock)` block and a nonblocking assignment.

The third element was a negative-edge-triggered D flip-flop. Its output Q_c was updated using an `always @(negedge Clock)` block.

The three implementations were designed to distinguish continuous data tracking during an active clock level from data capture at rising and falling clock edges.

2.4 Part IV: Eight-Bit Register and Seven-Segment Display

An eight-bit register was implemented in `main.v` to store a binary value from the DE10-Lite input switches. The register was declared as `reg [7:0] A` and updated through a sequential `always` block.

The eight switches `SW[7:0]` provided the input data. Pushbutton `KEY[1]` was used as the register clock, while `KEY[0]` provided an active-low asynchronous reset.

The register operation was defined as:

$$A_{\text{next}} = \begin{cases} 0, & \text{if } KEY[0] = 0, \\ SW[7:0], & \text{on rising edge of } KEY[1]. \end{cases} \tag{10}$$

Four instances of the `seg7_display` module were used to display the stored and current values in hexadecimal format.

The stored register value A was divided into two four-bit groups. The upper nibble $A[7:4]$ was connected to `HEX3`, and the lower nibble $A[3:0]$ was connected to `HEX2`.

Similarly, the current switch value was displayed directly on `HEX1` and `HEX0`.

The decoder in `seg7_display.v` used a combinational `case` statement to convert each four-bit input into the corresponding eight-bit seven-segment output pattern.

This arrangement allowed the stored value to remain unchanged while the switch inputs were modified, until another rising clock edge occurred.

3 Discussion

The implemented circuits demonstrated different methods of storing binary information in digital systems. The gated SR and D latches used combinational feedback to retain their states, while the

behavioral flip-flops used clock-edge sensitivity to control data capture.

The eight-bit register extended single-bit storage to a multi-bit application. Its asynchronous reset provided a method for clearing the stored value independently of the clock. The seven-segment decoder enabled the stored and current switch values to be represented simultaneously in hexadecimal notation.

4 Conclusion

This laboratory implemented fundamental sequential logic elements using Verilog HDL. The designs demonstrated level-sensitive latches, positive- and negative-edge-triggered flip-flops, and an eight-bit register with asynchronous reset. The final register design integrated sequential storage with combinational seven-segment decoding, illustrating how these components can be combined in an FPGA-based digital system.

Part I

```
module part1(
    input  Clk,
    input  R,
    input  S,
    output Q
);

    wire R_g, S_g;
    wire Qa, Qb /* synthesis keep */;

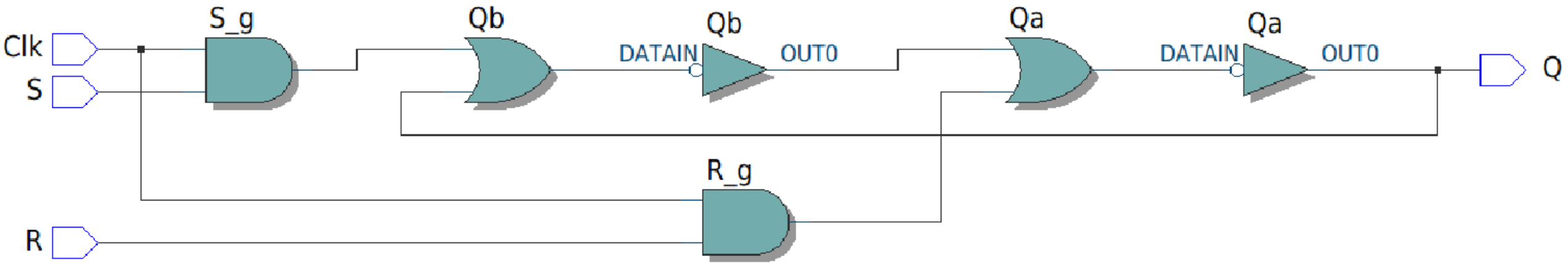
    // Gate the R and S inputs with Clk
    assign R_g = R & Clk;
    assign S_g = S & Clk;

    // Cross-coupled NOR gates
    assign Qa = ~(R_g | Qb);
    assign Qb = ~(S_g | Qa);

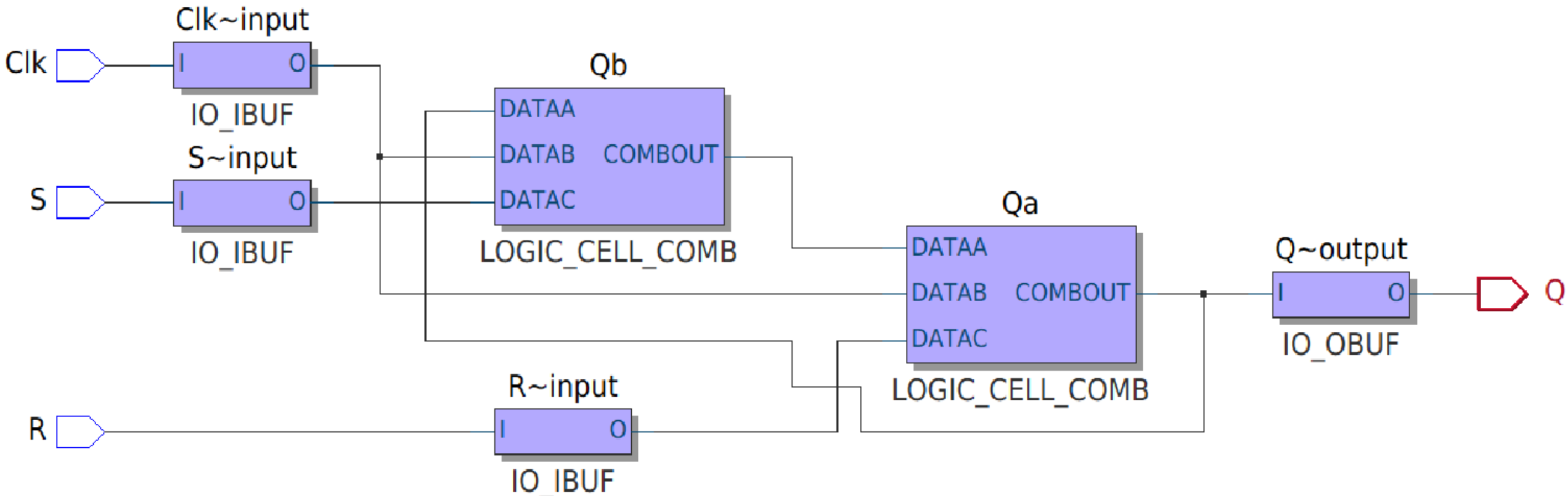
    // Latch output
    assign Q = Qa;

endmodule
```

RTL Schematic



TL Viewer


Clk~input
Clk
I
O
IO_IBUF
Qb
DATAA
DATAB
COMBOUT
DATAC
LOGIC_CELL_COMB
S~input
S
I
O
IO_IBUF
Qa
DATAA
DATAB
COMBOUT
DATAC
LOGIC_CELL_COMB
Q~output
I
O
IO_OBUF
Q
R~input
R
I
O
IO_IBUF

Part II

```
module part2(D, Clk, Q);

    input D, Clk;
    output Q;

    wire R, S_g, R_g, Qa, Qb /* synthesis keep */;

    // Generate S and R from D
    assign R = ~D;

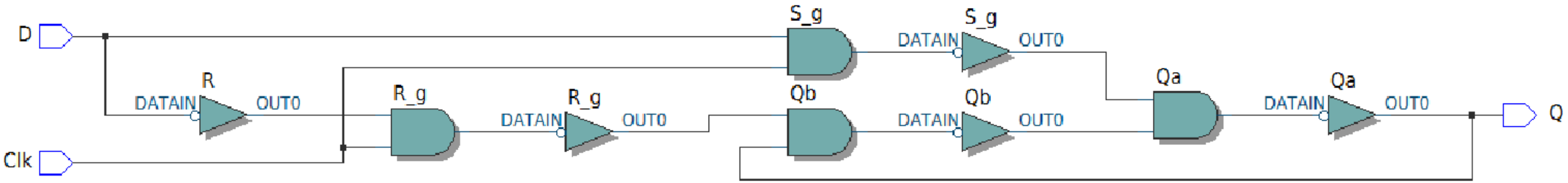
    // Gated active-low S and R signals
    assign S_g = ~(D & Clk);
    assign R_g = ~(R & Clk);

    // Cross-coupled NAND latch
    assign Qa = ~(S_g & Qb);
    assign Qb = ~(R_g & Qa);

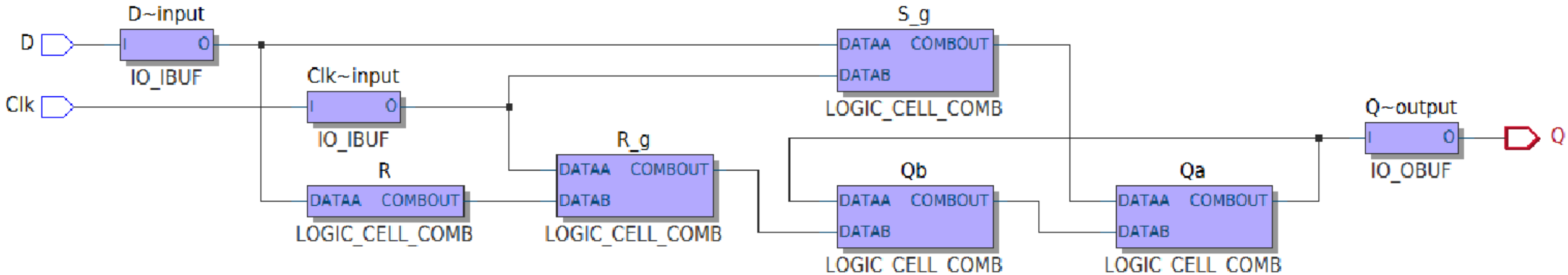
    // Output
    assign Q = Qa;

endmodule
```

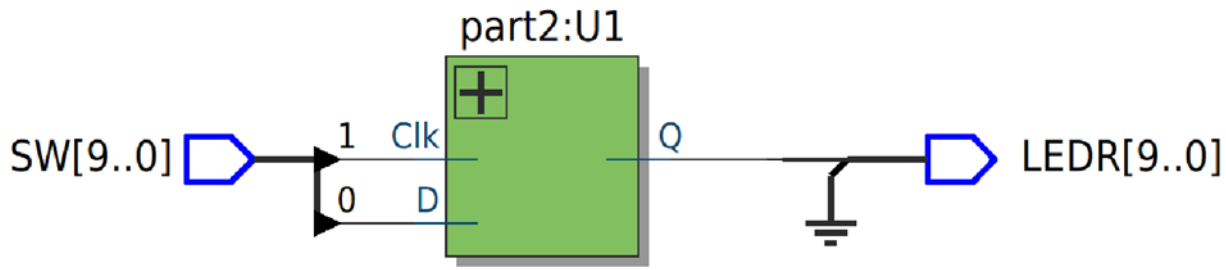
RTL Schematic

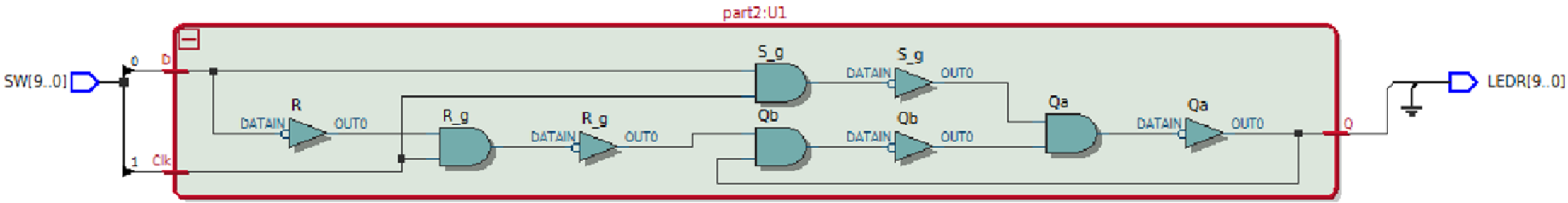


TL Viewer


D
Clk
D~input
I
O
IO_IBUF
Clk~input
I
O
IO_IBUF
R
DATAA
COMBOUT
LOGIC_CELL_COMB
R_g
DATAA
COMBOUT
DATAB
LOGIC_CELL_COMB
S_g
DATAA
COMBOUT
DATAB
LOGIC_CELL_COMB
Qb
DATAA
COMBOUT
DATAB
LOGIC_CELL_COMB
Qa
DATAA
COMBOUT
DATAB
LOGIC_CELL_COMB
Q~output
I
O
IO_OBUF
Q

Implementation on DE10Lite





Part III

```
module part3(
   input  D,
   input  Clock,
   output Q
);

   wire Qm;
   wire Clock_n;

   // Inverted clock for the master latch
   assign Clock_n = ~Clock;

   // Master latch
   part2 Master (
      .D   (D),
      .Clk (Clock_n),
      .Q   (Qm)
   );

   // Slave latch
   part2 Slave (
      .D   (Qm),
      .Clk (Clock),
      .Q   (Q)
   );

endmodule
```

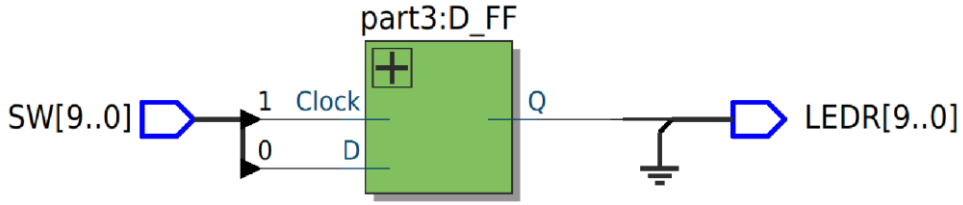
```
module main (
   input  [9:0] SW,
   output [9:0] LEDR
);

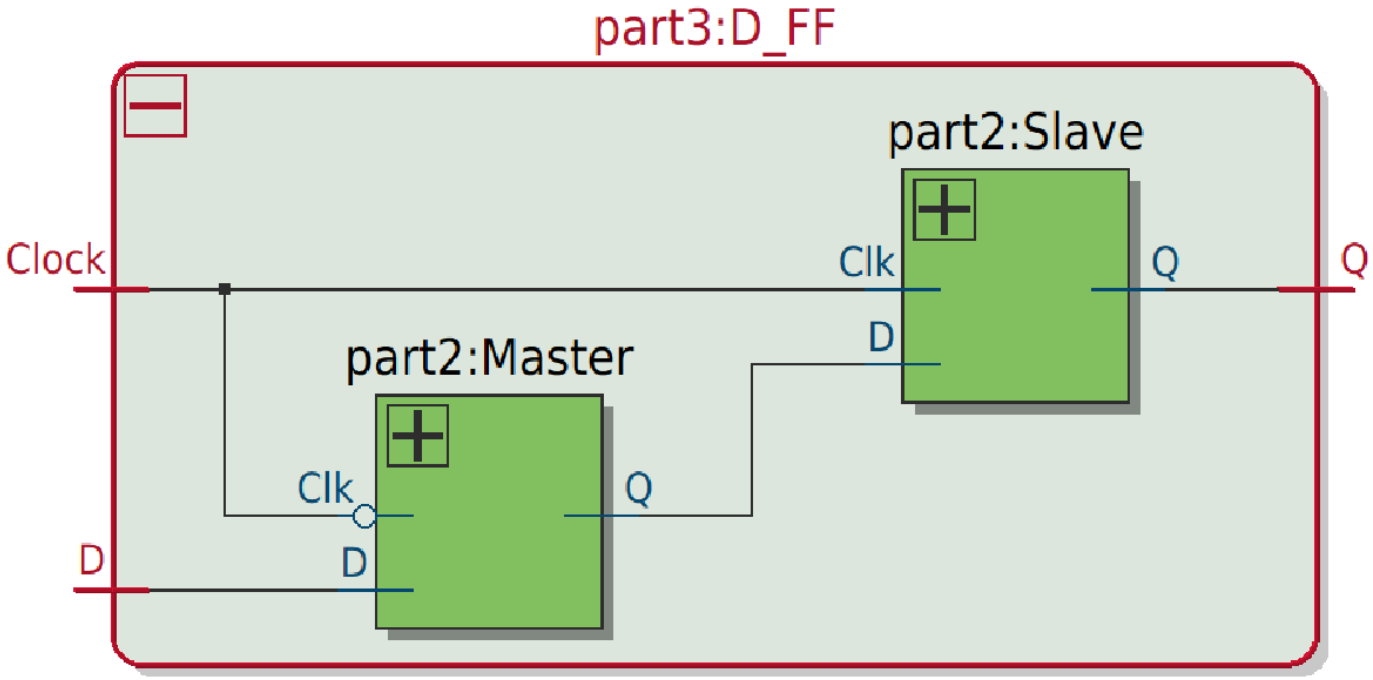
   wire Q;

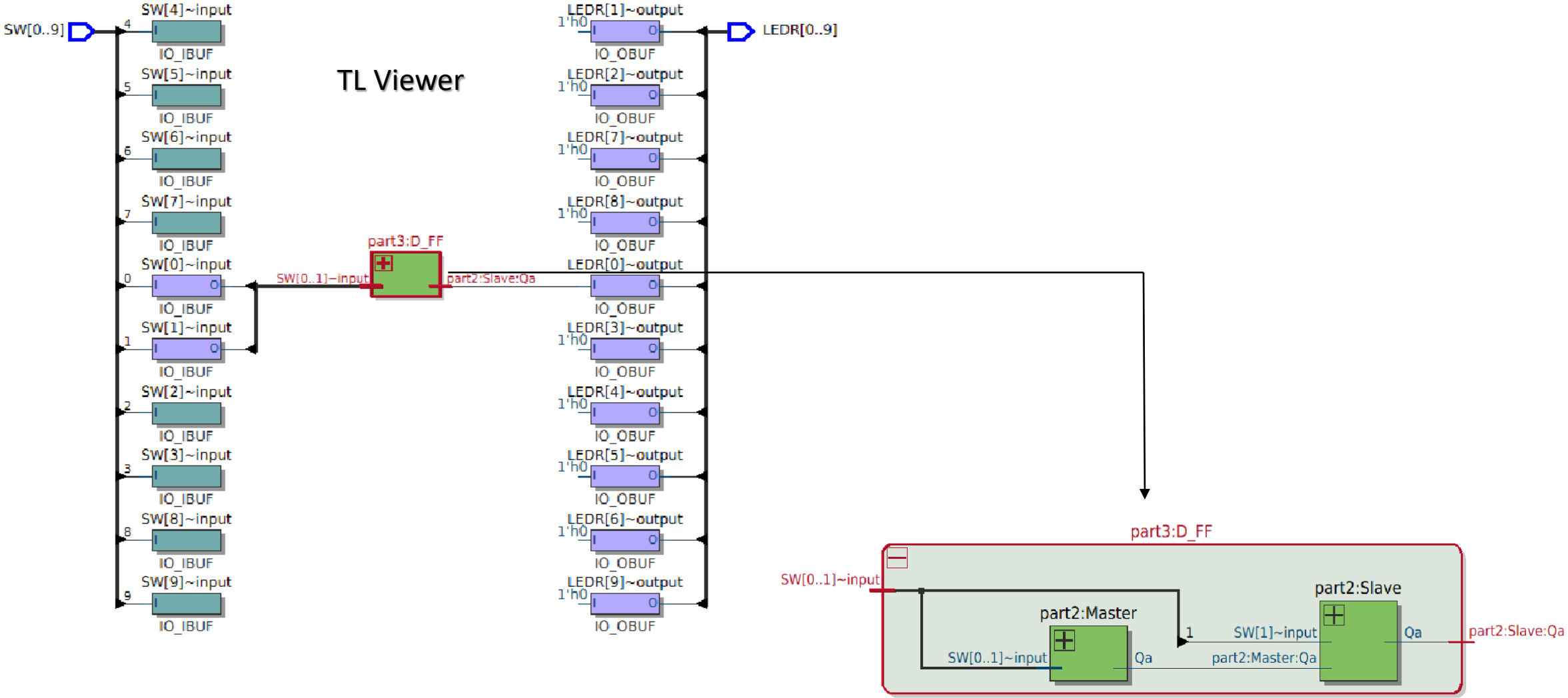
   part3 D_FF(
      .D     (SW[0]),
      .Clock (SW[1]),
      .Q     (Q)
   );

   assign LEDR[0] = Q;

endmodule
```






TL Viewer
SW[0..9]
SW[4]~input
SW[5]~input
SW[6]~input
SW[7]~input
SW[0]~input
SW[1]~input
SW[2]~input
SW[3]~input
SW[8]~input
SW[9]~input
IO_IBUF
part3:D_FF
SW[0..1]~input
part2:Slave:Qa
LEDR[1]~output
LEDR[2]~output
LEDR[7]~output
LEDR[8]~output
LEDR[0]~output
LEDR[3]~output
LEDR[4]~output
LEDR[5]~output
LEDR[6]~output
LEDR[9]~output
1'h0
IO_OBUF
LEDR[0..9]
part3:D_FF
SW[0..1]~input
part2:Master
SW[0..1]~input
Qa
1
part2:Slave
SW[1]~input
part2:Master:Qa
Qa
part2:Slave:Qa

Part IV

```
module main (
    input  wire D,
    input  wire Clock,
    output reg  Qa,
    output reg  Qb,
    output reg  Qc
);

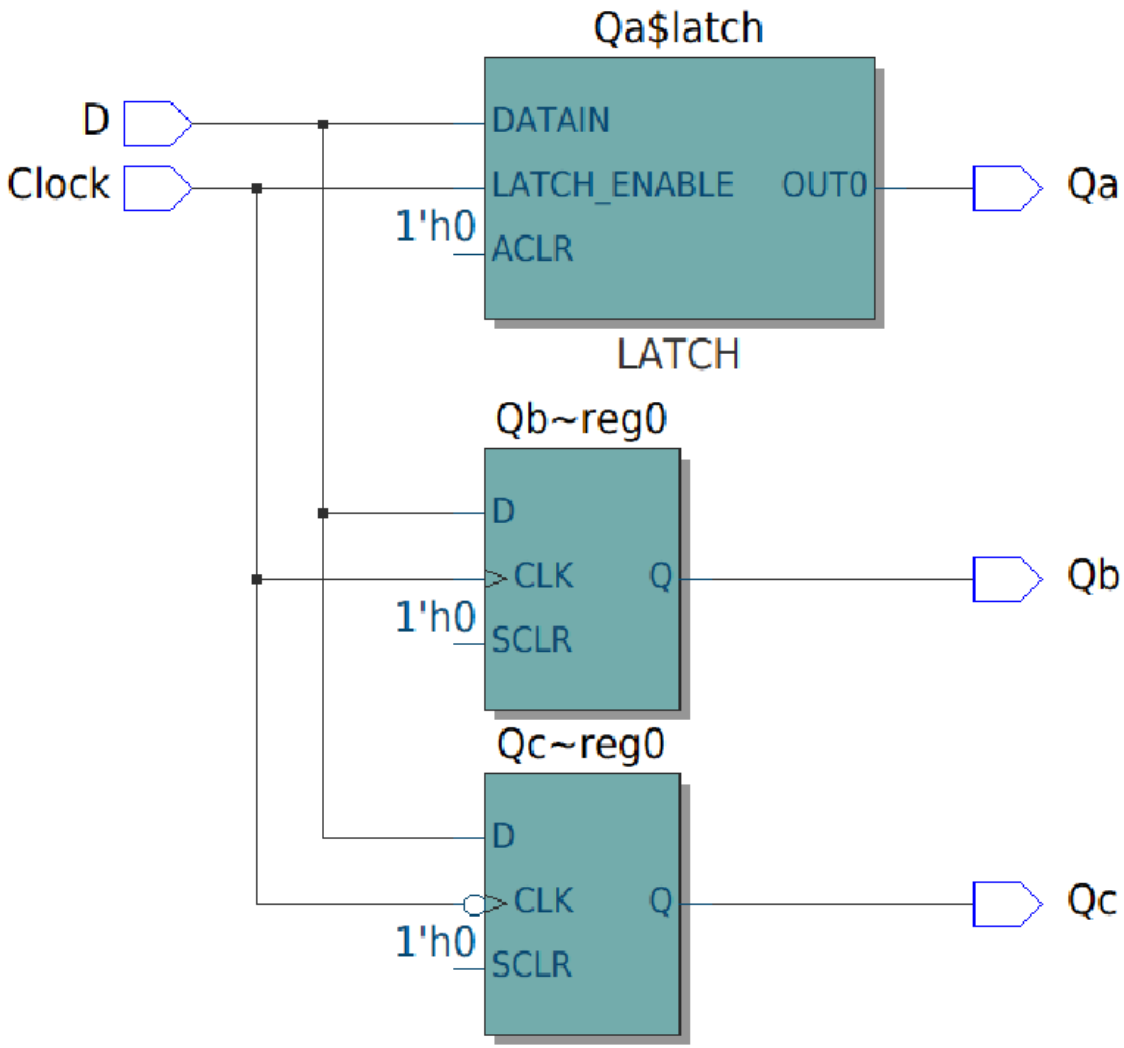
    // Gated D latch
    // Transparent when Clock = 1
    always @ (D or Clock)
    begin
        if (Clock)
            Qa = D;
    end

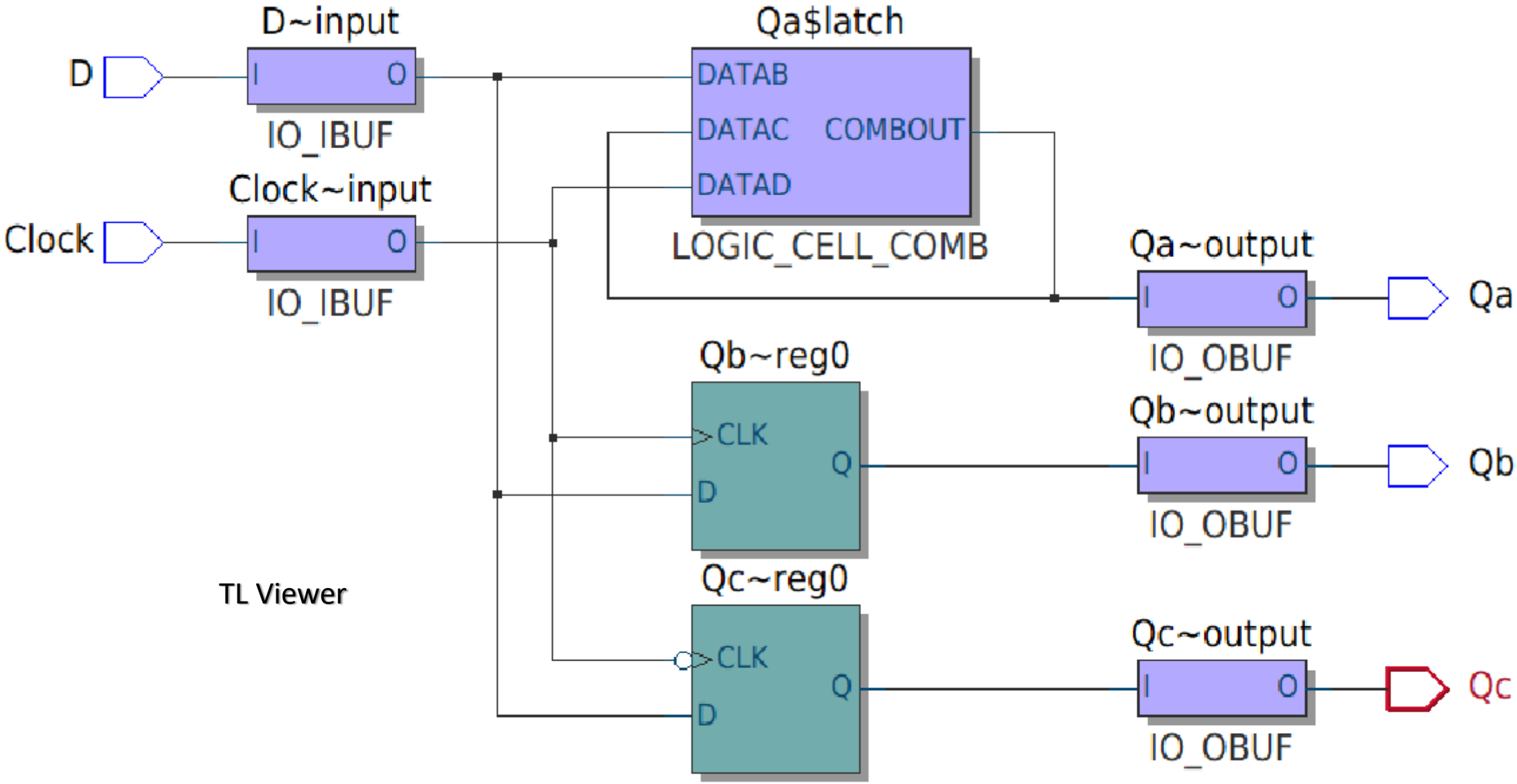
    // Positive-edge-triggered D flip-flop
    always @ (posedge Clock)
    begin
        Qb <= D;
    end

    // Negative-edge-triggered D flip-flop
    always @ (negedge Clock)
    begin
        Qc <= D;
    end

endmodule
```

RTL Schematic




D~input
D
I
O
IO_IBUF
Clock~input
Clock
I
O
IO_IBUF
Qa$latch
DATAB
DATAC
COMBOUT
DATAD
LOGIC_CELL_COMB
Qa~output
I
O
IO_OBUF
Qa
Qb~reg0
CLK
Q
D
Qb~output
I
O
IO_OBUF
Qb
Qc~reg0
CLK
Q
D
Qc~output
I
O
IO_OBUF
Qc

TL Viewer

Part V

```
module main (
    input  [7:0] SW,
    input  [1:0] KEY,
    output [7:0] HEX0,
    output [7:0] HEX1,
    output [7:0] HEX2,
    output [7:0] HEX3
);

    reg [7:0] A;

    // Store A
    // KEY0: active-low asynchronous reset
    // KEY1: clock
    always @(posedge KEY[1] or negedge KEY[0]) begin
        if (!KEY[0])
            A <= 8'b00000000;
        else
            A <= SW[7:0];
    end

    // Stored A -> HEX3 and HEX2
    seg7_display display_A_high (
        .bin_number(A[7:4]),
        .seg_display(HEX3)
    );

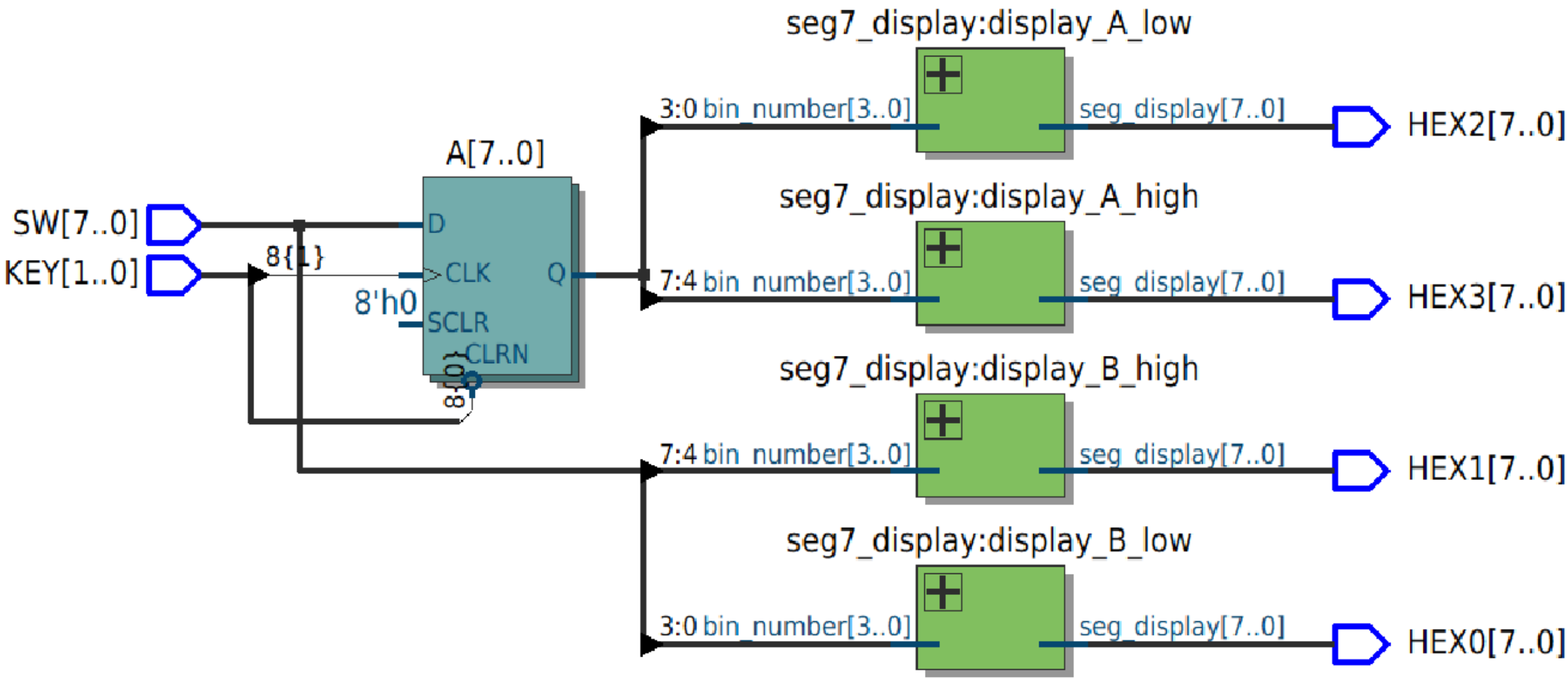
    seg7_display display_A_low (
        .bin_number(A[3:0]),
        .seg_display(HEX2)
    );

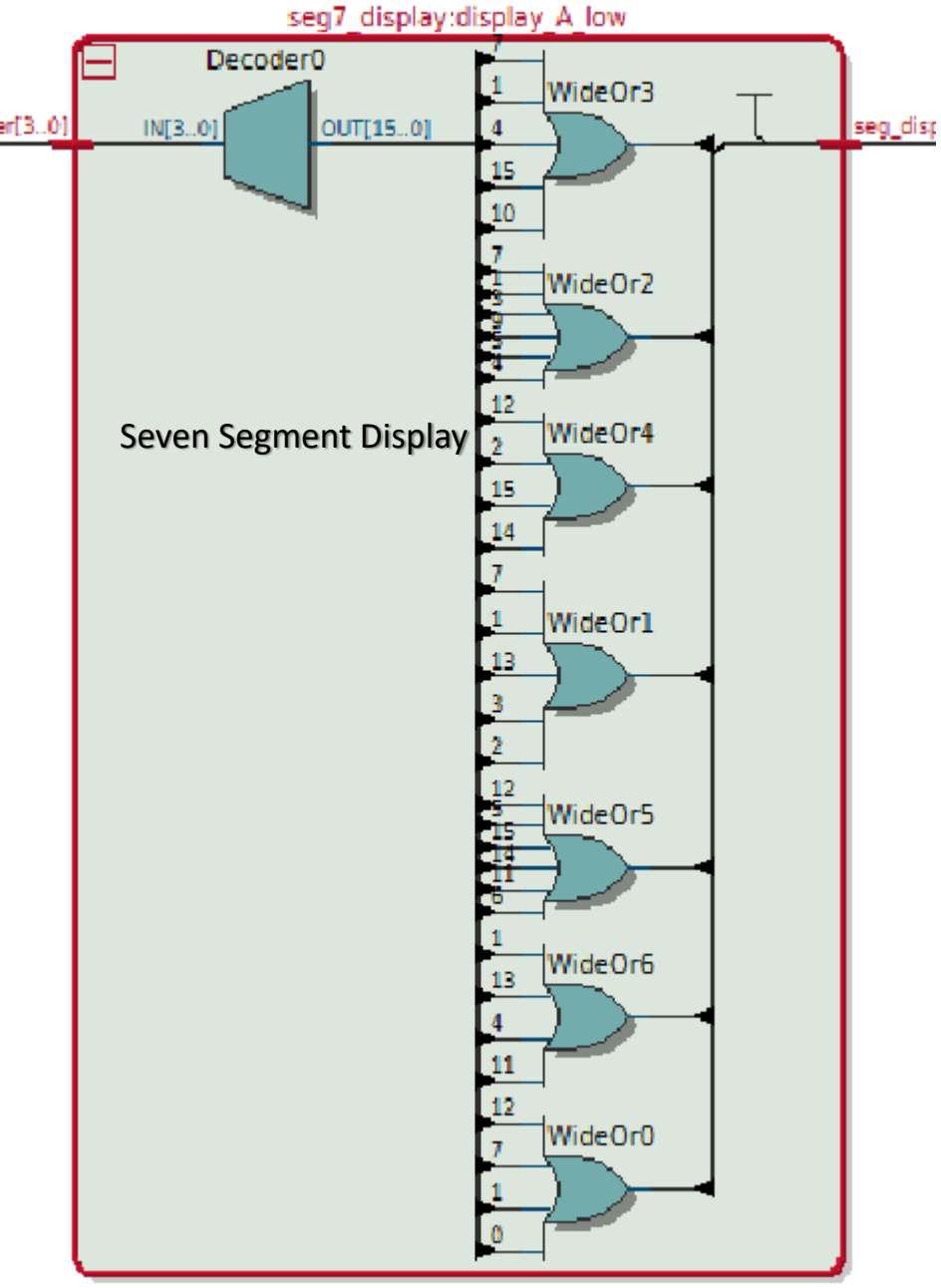
    // Current switch value B -> HEX1 and HEX0
    seg7_display display_B_high (
        .bin_number(SW[7:4]),
        .seg_display(HEX1)
    );

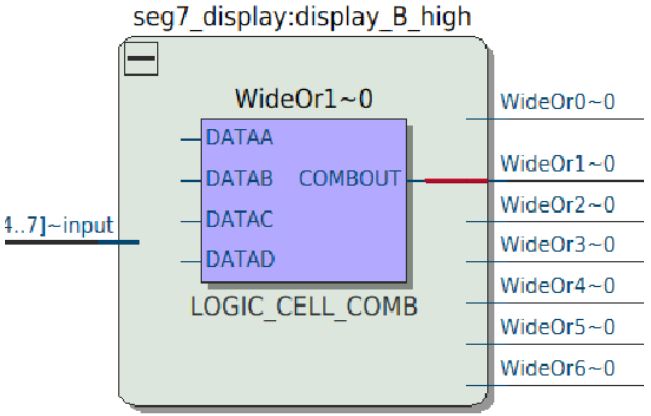
    seg7_display display_B_low (
        .bin_number(SW[3:0]),
        .seg_display(HEX0)
    );

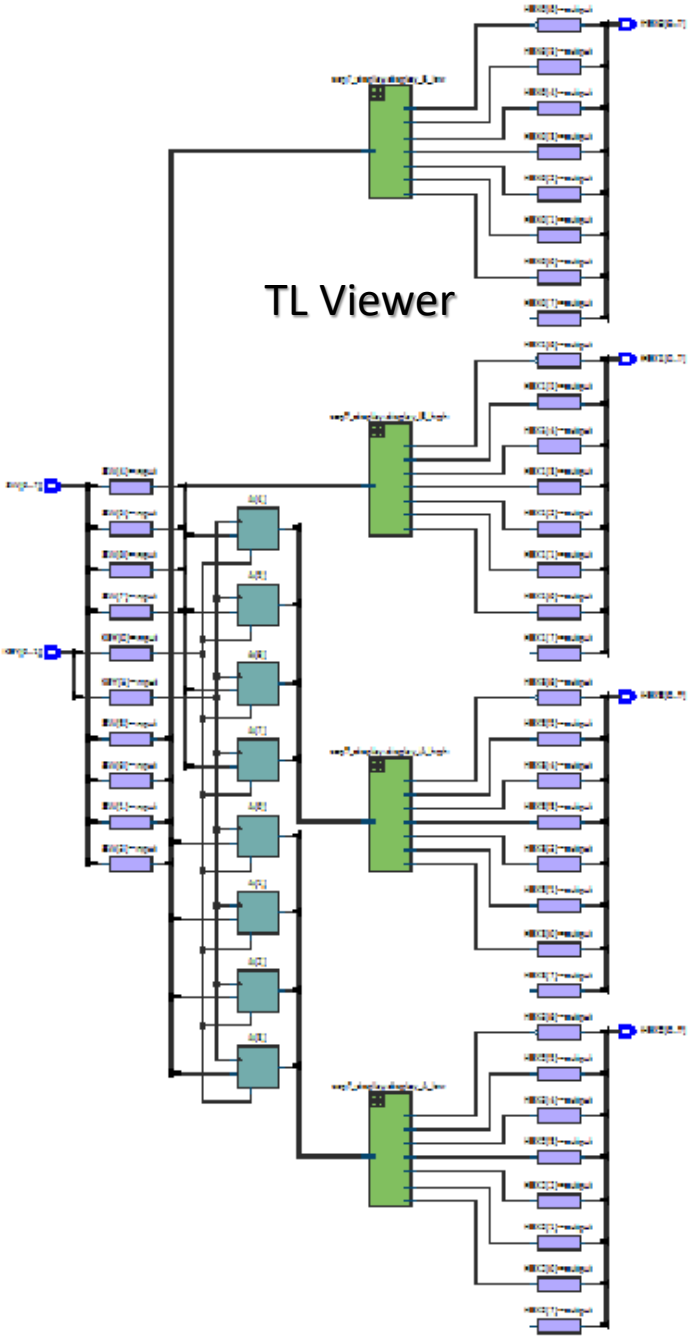
endmodule
```

RTL


seg7_display:display_A_low
3:0 bin_number[3..0]
seg_display[7..0]
HEX2[7..0]
A[7..0]
SW[7..0]
KEY[1..0]
8{1}
D
CLK
Q
8'h0
SCLR
CLRN
8{0}
seg7_display:display_A_high
7:4 bin_number[3..0]
seg_display[7..0]
HEX3[7..0]
seg7_display:display_B_high
7:4 bin_number[3..0]
seg_display[7..0]
HEX1[7..0]
seg7_display:display_B_low
3:0 bin_number[3..0]
seg_display[7..0]
HEX0[7..0]


seg7_display:display_A_low
Decoder0
IN[3..0]
OUT[15..0]
WideOr3
WideOr2
WideOr4
WideOr1
WideOr5
WideOr6
WideOr0
Seven Segment Display


seg7_display:display_B_high
WideOr1~0
DATAA
DATAB
DATAC
DATAD
COMBOUT
LOGIC_CELL_COMB
WideOr0~0
WideOr1~0
WideOr2~0
WideOr3~0
WideOr4~0
WideOr5~0
WideOr6~0


TL Viewer